

Laporan Praktikum Kontrol Cerdas

Minggu ke - 6

Nama : Maheswara Aryasatya Nugraha
NIM : 224308036
Kelas : TKA-6B
Akun Github : <https://github.com/maheswaraaryasatya>
Student Lab Assistant : Mas Alfian

1. Judul Percobaan: ***Canny Edge Detection & Lane Detection with Instance Segmentation***

2. Tujuan Percobaan:

- Memahami konsep *Canny Edge Detection* sebagai metode dasar deteksi tepi.
- Menggunakan *Instance Segmentation* untuk deteksi jalur rel kereta (Lane Detection).
- Menggunakan dataset *Rail Segmentation* dari Kaggle untuk eksperimen.
- Menggabungkan metode *Canny Edge Detection* dengan *Instance Segmentation* untuk meningkatkan deteksi jalur.

3. Landasan Teori:

Canny Edge Detection atau deteksi tepi adalah langkah penting dalam proses pengolahan citra yang bertujuan untuk mengidentifikasi batas-batas atau kontur objek dalam gambar, seperti Sobel, Prewitt, dan Roberts telah lama digunakan dalam dunia pengolahan citra. Namun, metode *Canny Edge Detection* dianggap lebih unggul karena mampu mendeteksi tepi dengan akurasi yang lebih tinggi dan menghasilkan gambar dengan tingkat kebisingan yang rendah (Luh Putu Risma Noviana dkk., 2024). Keunggulan *Canny Edge Detection* dibandingkan dengan deteksi tepi lainnya yaitu *good detection*, memaksimalkan *Signal to Noise Ration* (SNR) agar semua tepi dapat terdeteksi dengan baik, kemudian *good location*, untuk meminimalkan jarak deteksi tepi yang sebenarnya dengan tepi yang dihasilkan melalui pemrosesan, sehingga

lokasi tepi terdeteksi menyerupai tepi secara nyata (semakin besar nilai *Loc*, maka semakin besar kualitas deteksi yang dimiliki), selanjutnya ada *one respon to single edge*, untuk menghasilkan tepi tunggal /tidak memberikan tepi yang bukan tepi sebenarnya (**Perangin-angin dkk., 2019**). Pada praktikum ini juga menggunakan proses *Instance Segmentation* untuk deteksi jalur rel kereta (*Lane Detection*). *Instance Segmentation* atau proses anotasi merupakan proses pemberian label pada suatu gambar, biasanya menggunakan *tools Roboflow* (**Wulanningrum dkk., 2024**). Pelatihan datanya menggunakan kurang lebih 2000 foto rel kereta api dengan berbagai macam *track*, ada yang lurus, berbelok, *single track*, *double track*, dan lain-lain. Setiap *epoch* melibatkan pemasukan dataset ke dalam model, penyesuaian parameter model, dan pembaruan bobot untuk mengoptimalkan kinerja model dalam mendeteksi dan segmentasi rel kereta api dengan akurat. Proses iteratif ini bertujuan untuk meminimalkan perbedaan antara rel kereta api yang diprediksi dengan objek lain dan anotasi kebenaran dasar dalam dataset pelatihan. Teknik augmentasi data diterapkan selama pelatihan untuk meningkatkan kemampuan generalisasi model (**Husain dkk., 2024**). Algoritma ini tahan terhadap oklusi yang kompleks, sehingga sangat berguna untuk adaptasi terhadap lingkungan AV. Algoritma ini telah digabungkan oleh arsitektur jaringan klasifikasi seperti ENet, Segnet, dan Resnet38 yang dibuat khusus untuk identifikasi gambar. Hasilnya adalah arsitektur Resnet38 memiliki akurasi rata-rata yang paling tinggi, namun membutuhkan memori yang lebih besar (**Darwin & Saputro, 2021**).

4. Analisis dan Diskusi:

- Analisis

Kode yang diberikan mengimplementasikan kombinasi metode Canny Edge Detection dan model YOLO untuk melakukan deteksi tepi sekaligus deteksi objek atau segmentasi. Pada tahap awal, program memuat model YOLO dari Ultralytics, memastikan bahwa model tersedia sebelum melanjutkan ke pemrosesan video secara real-time menggunakan OpenCV. Kamera diakses dengan `cv2.VideoCapture(0)`, dan jika terjadi kegagalan, program akan memberikan peringatan dan keluar.

Setiap frame yang ditangkap diproses dalam beberapa tahap, dimulai dengan konversi ke skala abu-abu sebelum diterapkan metode Canny Edge Detection dengan ambang batas bawah 50 dan atas 150 untuk mengekstrak tepi objek. Pada saat yang sama, frame asli dikonversi ke format RGB sebelum dimasukkan ke dalam model YOLO untuk prediksi objek.

Jika model adalah model segmentasi, area objek ditandai dengan poligon biru transparan menggunakan `cv2.fillPoly()`, sementara jika model adalah model deteksi objek, bounding box digunakan untuk mengelilingi objek yang terdeteksi, dengan tambahan teks label jika tersedia.

Selanjutnya, program menggunakan masking untuk menandai area objek berdasarkan hasil prediksi YOLO, yang kemudian digunakan untuk membedakan warna tepi yang terdeteksi oleh metode Canny, di mana tepi dalam area objek yang terdeteksi oleh YOLO diwarnai merah, sementara tepi di luar objek tetap putih.

Akhirnya, dua gambar hasil (frame dengan anotasi YOLO dan hasil deteksi tepi) ditampilkan secara berdampingan menggunakan `np.hstack()`, memungkinkan pengguna untuk melihat kombinasi dari kedua metode dalam satu tampilan, dengan opsi untuk keluar dari program dengan menekan tombol 'q'.

- Diskusi

Kode ini menawarkan solusi menarik dalam menggabungkan metode klasik Canny Edge Detection dengan pendekatan deep learning menggunakan YOLO, menciptakan sistem yang mampu mendeteksi tepi sekaligus mengenali dan menyoroti objek yang terdeteksi. Namun, ada beberapa aspek yang dapat ditingkatkan agar sistem lebih optimal, terutama dalam hal performa, karena model YOLO membutuhkan daya komputasi yang cukup tinggi, terutama jika dijalankan di CPU; penggunaan GPU bisa menjadi solusi untuk mempercepat pemrosesan.

Selain itu, ukuran frame bisa dikurangi untuk mengurangi beban komputasi tanpa kehilangan terlalu banyak informasi penting. Dalam hal segmentasi, warna overlay yang lebih transparan dapat meningkatkan keterbacaan gambar, sementara penghalusan batas segmentasi bisa membantu mengurangi artefak yang muncul di sekitar objek yang terdeteksi.

Dari sisi deteksi tepi, hasil dari metode Canny bisa mengalami noise yang cukup tinggi, terutama pada gambar dengan banyak detail atau tekstur, sehingga penggunaan Gaussian Blur tambahan sebelum deteksi tepi dapat membantu mengurangi gangguan yang tidak perlu.

Selain itu, kode saat ini hanya menampilkan hasil dalam jendela OpenCV tanpa menyimpan data, sehingga menambahkan fitur penyimpanan hasil dalam format gambar atau video bisa bermanfaat untuk keperluan analisis lebih lanjut.

Secara keseluruhan, sistem ini memiliki potensi besar untuk berbagai aplikasi dunia nyata seperti deteksi jalur kereta api, pengawasan lalu lintas untuk kendaraan dan pejalan kaki, serta dalam bidang medis untuk analisis citra seperti mendeteksi batas sel pada mikroskop. Namun, sistem ini masih memerlukan penyempurnaan dalam aspek kecepatan pemrosesan, akurasi deteksi, serta fleksibilitas dalam menghadapi kondisi pencahayaan dan lingkungan yang berbeda. Dengan eksplorasi lebih lanjut dan optimasi model, metode gabungan ini dapat diterapkan dalam berbagai skenario praktis, termasuk keamanan, industri otomotif, serta bidang penelitian ilmiah.

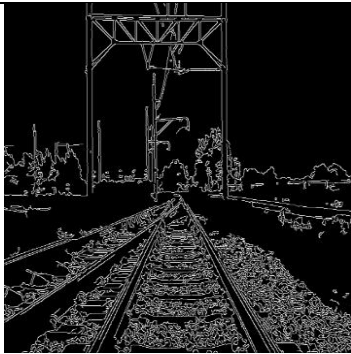
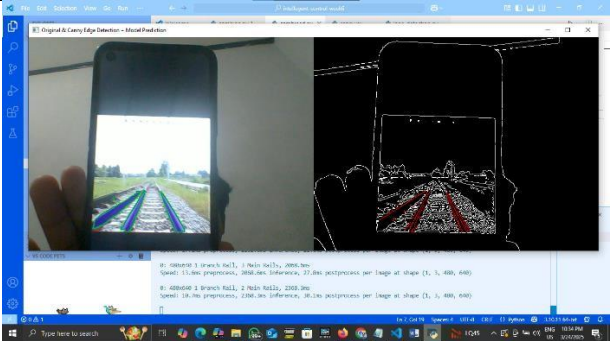
5. Assignment:

Berdasarkan praktikum keenam yang telah dilakukan dari kode program ini adalah membuat sebuah sistem yang dapat melakukan deteksi tepi menggunakan metode *Canny Edge Detection* dan mengombinasikannya dengan deteksi objek atau segmentasi menggunakan model YOLO dari Ultralytics untuk menyoroti objek yang terdeteksi dalam sebuah video yang diambil secara real-time dari kamera. Program dimulai dengan memuat model YOLO dari path yang telah ditentukan menggunakan

YOLO(MODEL_PATH), memastikan bahwa model tersedia sebelum melanjutkan ke pemrosesan video menggunakan OpenCV. Setelah kamera diakses melalui **cv2.VideoCapture(0)**, setiap frame yang ditangkap dikonversi ke skala abu-abu dengan **cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)** agar dapat dilakukan deteksi tepi menggunakan metode *Canny Edge Detection*, yang menghasilkan gambar biner dengan tepi putih pada latar belakang hitam melalui **cv2.Canny(gray, low_threshold, high_threshold)**. Secara paralel, frame asli dikonversi ke format RGB dengan **cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)**, kemudian diproses oleh model YOLO untuk melakukan prediksi objek atau segmentasi, di mana hasil prediksi pertama disimpan dalam **predictions = results[0]**. Jika model menggunakan segmentasi, maka program akan menggambar area objek menggunakan poligon biru transparan dengan **cv2.fillPoly(overlay, [mask], (255, 0, 0))**, sedangkan jika model hanya mendeteksi objek dalam bentuk bounding box, maka objek akan dilingkari dengan kotak hijau menggunakan **cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)**, serta label objek akan ditampilkan jika tersedia menggunakan **cv2.putText()**. Selanjutnya, masking digunakan untuk membedakan tepi objek yang terdeteksi oleh YOLO dengan tepi yang dideteksi oleh metode Canny, di mana tepi dalam area objek yang dikenali YOLO akan diwarnai merah, sementara tepi lainnya tetap berwarna putih dengan **edges_bgr[np.where((mask_model == 255) & (edges == 255))] = [0, 0, 255]**. Kedua hasil, yaitu frame dengan anotasi YOLO dan hasil deteksi tepi, ditampilkan secara berdampingan menggunakan **np.hstack()** untuk memberikan tampilan visual yang lebih jelas mengenai perbedaan antara tepi umum dan tepi dari objek yang dikenali. Program berjalan dalam *loop* hingga pengguna menekan tombol 'q', yang akan memicu **cv2.waitKey(1) & 0xFF == ord('q')** untuk keluar, setelah itu kamera akan dilepaskan dengan **cap.release()** dan semua jendela yang terbuka akan ditutup dengan **cv2.destroyAllWindows()** untuk mencegah kebocoran memori. Dengan kombinasi dari metode deteksi tepi tradisional dan teknologi deep learning untuk segmentasi atau deteksi objek, program ini dapat digunakan untuk berbagai aplikasi seperti deteksi rel, pengawasan lalu lintas, dan lainnya.

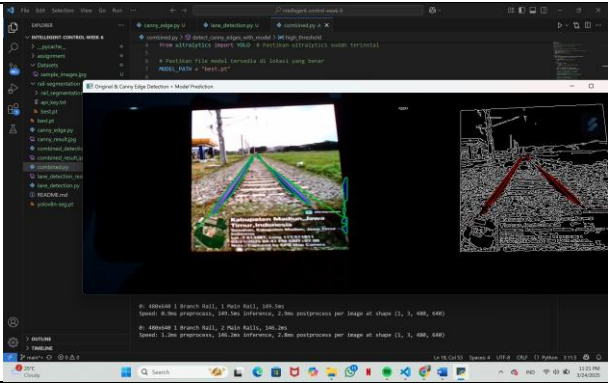
6. Data dan Output Hasil Pengamatan:

- Data Sebelum Modifikasi

No.	Variabel	Hasil Pengamatan
1.	Gambar asli	
2.	Canny Edge Detection	
3.	Lane Detection dengan Instance Segmentation (YOLOv8-seg)	
4.	Menggabungkan Canny Edge Detection dengan Lane Detection	

- Data Sesudah Modifikasi

No.	Variabel	Hasil Pengamatan
1.	Gambar Asli	
2.	Canny Edge Detection	
3.	Lane Detection dengan Instance Segmentation (YOLOv8-seg)	

4.	Menggabungkan Canny Edge Detection dengan Lane Detection	 The screenshot displays a Python script in an IDE. The script performs the following steps: - Imports <code>cv2</code> and <code>ximgproc</code> from <code>cv2</code>. - Defines a function <code>detect_lane_lines</code> that takes an image <code>img</code> as input. - Converts the image to grayscale using <code>cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)</code>. - Applies Canny edge detection using <code>cv2.Canny</code>. - Uses <code>ximgproc.houghLinesP</code> to detect lines in the image. - Filters the detected lines based on their slope and length. - Returns the filtered lines. The output of the script is shown in two side-by-side images: the original image with green lines representing Canny edges, and the same image with red lines representing the detected lane boundaries.
----	---	--

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- Program berhasil mengombinasikan metode *Canny Edge Detection* dengan model YOLO untuk mendeteksi tepi sekaligus mengenali dan menyoroti objek dalam video real-time.
- Model YOLO mampu mendeteksi objek dengan baik, baik dalam bentuk bounding box maupun segmentasi, yang kemudian ditandai dengan warna dan anotasi yang sesuai.
- Teknik masking digunakan untuk membedakan tepi objek yang terdeteksi YOLO dengan tepi umum dari hasil *Canny Edge Detection*, memungkinkan visualisasi yang lebih jelas.
- Program dapat berjalan dalam *loop* hingga pengguna menekan tombol keluar, dengan pengelolaan memori yang baik untuk menghindari kebocoran sumber daya.

8. Saran:

Untuk meningkatkan performa dan akurasi, program dapat dioptimalkan dengan penggunaan GPU agar prediksi YOLO berjalan lebih cepat dan responsif. Selain itu, preprocessing tambahan seperti *Gaussian Blur* sebelum *Canny Edge Detection* dapat membantu mengurangi noise yang tidak diperlukan. Untuk meningkatkan keterbacaan visual, transparansi overlay warna dapat disesuaikan agar objek yang terdeteksi lebih jelas tanpa menutupi detail penting dalam gambar. Menambahkan fitur penyimpanan hasil deteksi dalam bentuk gambar atau video bisa menjadi nilai tambah untuk analisis lebih lanjut di berbagai aplikasi dunia nyata.

9. Daftar Pustaka:

- Darwin, S., & Saputro, N. (2021). *SURVEI APLIKASI SEGMENTASI CITRA UNTUK AUTONOMOUS VEHICLE*.
- Husain, B. H., Osawa, T., Astiti, S. P. C., Frianto, D., Nandika, M. R., & Augustine, D. (2024). Deteksi Lubang Jalan Secara Otomatis dari Rekaman Drone Menggunakan Model Machine Learning Berbasis YOLOv5 Instance Segmentation di Kota Pekanbaru, Provinsi Riau, Indonesia. *Jurnal Zona*, 8(2), 137–146. <https://doi.org/10.52364/zona.v8i2.126>
- Luh Putu Risma Noviana, I Putu Eka Indrawan, & Gde Iwan Setiawan. (2024). ANALYSIS OF CANNY EDGE DETECTION METHOD FOR FACIAL RECOGNITION IN DIGITAL IMAGE PROCESSING. *Jurnal Manajemen dan Teknologi Informasi*, 15(2), 29–34. <https://doi.org/10.59819/jmti.v15i2.4107>
- Perangin-angin, R., Harianja, E. J. G., & No, J. H. T. (2019). *COMPARISON DETECTION EDGE LINES ALGORITMA CANNY DAN SOBEL*. 2.
- Wulanningrum, R., Handayani, A. N., & Wibawa, A. P. (2024). Perbandingan Instance Segmentation Image Pada Yolo8. *Jurnal Teknologi Informasi dan Ilmu Komputer*, 11(4), 753–760. <https://doi.org/10.25126/jtiik.1148288>